Claude Certified Developer – Foundations (CCDV-F)

Answer key · D1 / S3

53 questions · Time: 120 minutes · English

Answer summary

#	Correct answer	Domain	Skill	Level
1	**A, B, C**	D2	Software Engineering Foundations	hard
2	**A, B**	D8	Tool Implementation	medium
3	**C**	D5	LLM Fundamentals	easy
4	**B**	D1	Agent Patterns & Frameworks	medium
5	**A, B**	D3	Claude Code	medium
6	**A**	D5	Technical Fundamentals	easy
7	**C, D**	D2	Claude Application Design	medium
8	**A**	D8	Agentic Customization	easy
9	**B, C**	D1	Agent Construction with Claude	medium
10	**A**	D2	Understanding Requirements	easy
11	**D, E**	D3	Claude Code	medium
12	**B, D, E**	D5	Cost and Token Management	hard
13	**A, B**	D2	Claude Application Design	medium
14	**A, B, C**	D6	Context Engineering	hard
15	**B, E**	D7	Identity, Secrets and Key Management	easy
16	**B**	D6	Prompt Engineering	medium
17	**C**	D7	AI Application Security	easy
18	**A, D**	D8	Agentic Customization	medium
19	**A, E**	D6	Output Handling	medium
20	**B, D**	D2	Understanding Requirements	medium
21	**C**	D2	Configuration Management	hard
22	**C**	D8	MCP Server Development	hard
23	**B**	D5	Cost and Token Management	medium
24	**D**	D2	Systems Life Cycle	hard
25	**A, D**	D1	Agent Patterns & Frameworks	medium
26	**B**	D5	LLM Fundamentals	hard
27	**C**	D7	Claude Hooks	medium
28	**B, D**	D8	Tool Implementation	hard
29	**D**	D4	Debugging & Error Handling	hard
30	**B**	D8	Tool Implementation	medium
31	**B**	D2	Claude API Mechanics	medium
32	**A**	D1	Agent Architecture	easy
33	**A, C**	D5	LLM Fundamentals	medium
34	**A, C**	D5	Technical Fundamentals	medium
35	**B**	D1	Agent Patterns & Frameworks	easy
36	**A**	D5	Technical Fundamentals	medium
37	**A, D, E**	D2	Software Engineering Foundations	medium
38	**A**	D1	Agent Construction with Claude	easy
39	**B**	D2	Claude Application Design	easy

#	Correct answer	Domain	Skill	Level
40	**C**	D5	Model Selection and Tradeoffs	easy
41	**D**	D2	Software Engineering Foundations	easy
42	**A**	D2	Claude Application Design	easy
43	**A, C, D**	D2	Claude API Mechanics	medium
44	**D**	D2	Claude API Mechanics	easy
45	**A**	D6	Output Handling	easy
46	**A**	D6	Context Engineering	medium
47	**D**	D2	Software Engineering Foundations	easy
48	**C**	D2	Configuration Management	easy
49	**D**	D6	Prompt Engineering	easy
50	**A, C, E**	D7	AI Application Security	hard
51	**D**	D2	Claude Application Design	easy
52	**C**	D1	Agent Architecture	easy
53	**D**	D1	Agent Construction with Claude	hard

Answer key

Question 1 · D2 Applications & Integration · Software Engineering Foundations · hard

A team's CI calls the real Claude API on every pull request: the pipeline is slow, expensive and flaky. The team wants to test the integration code reliably. Select 3 suitable practices.

- **A.** **Use, in the PR pipeline, API client test doubles with recorded realistic responses, including tool_use, 429/529 errors and error events in the stream** [CORRECT]
- **B.** **Measure the quality of model responses in a separate evals suite, since doubles only verify the integration code and not model behavior** [CORRECT]
- **C.** **Periodically run a small contract suite against the real API to check that the doubles still reflect the real format, opening a fix task if they diverge** [CORRECT]
- **D.** Write doubles with only a hand-made happy-path JSON, since there will never be a need to compare them with real responses
- **E.** Keep real calls on every PR and rerun the job until it is green, since the flakiness is just noise

Correct answer: A, B, C. Correct (A, C and B): realistic doubles make PR tests fast and deterministic; periodic contract tests detect divergence between double and service without blocking unrelated PRs; and model quality is checked by evals, not by mocks. D is wrong because unverified hand-made doubles drift from reality and give false confidence. E is wrong because rerunning until it passes hides failures, keeps cost and slowness, and normalizes flaky tests.
Source: https://martinfowler.com/bliki/ContractTest.html

Question 2 · D8 Tools & MCPs · Tool Implementation · medium

Your update_ticket tool has the status parameter as a free string. In production Claude sends "Done", "finished" and "fechado", but the API only accepts open, in_progress and closed. Which changes to the tool definition address the cause? Select 2.

- **A.** **Declare status with "enum": ["open", "in_progress", "closed"] in the input_schema and explain each value in the parameter description.** [CORRECT]
- **B.** **Add input_examples with valid calls showing the accepted status values.** [CORRECT]
- **C.** Use forced tool_choice of type tool, so Claude always calls update_ticket.
- **D.** Cut the tool description to one short sentence, so as not to confuse the model.
- **E.** Silently map any unknown value to "open" in the handler, so it never fails.

Correct answer: A, B. Correct (A and B): an enum with descriptions closes the set of values, and input_examples (which must be valid per the input_schema) show correct usage. E hides the error and corrupts data. C forces the tool call but does not constrain the parameter's values. D goes against best practice: detailed descriptions are the most important factor in tool performance.
Source: https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools

Question 3 · D5 Model Selection & Optimization · LLM Fundamentals · easy

A dev enables thinking with display "omitted" so responses carry no reasoning text, believing this stops thinking from being billed. What actually happens to billing?

- **A.** Reasoning is billed as input tokens, at a lower rate than output.
- **B.** Only the visible text is billed; since the thinking field comes back empty, reasoning tokens cost nothing.
- **C.** **The thinking tokens actually generated are billed as output regardless of display; the breakdown appears in usage.output_tokens_details.thinking_tokens.** [CORRECT]
- **D.** Only a summary of the reasoning is billed, proportional to the summarized text length.

Correct answer: C. Correct (C): display only changes what is returned; generated reasoning is billed as output and output_tokens is the authoritative total. B is wrong because omitting does not zero the cost. A is wrong because generated thinking is output, not input. D is wrong because billing reflects the full reasoning, not the visible summary.
Source: https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost

Question 4 · D1 Agents & Workflows · Agent Patterns & Frameworks · medium

An Agent SDK agent has three custom tools that are all lookups (lookup_customer, lookup_order, lookup_inventory). When Claude requests all three in the same turn, they run one after another and latency triples. What is the best fix?

- **A.** Add the three tools to allowed_tools so the SDK treats them as safe for concurrent execution.
- **B.** **Mark the tools as read-only through readOnlyHint in their annotations, since custom tools run sequentially by default.** [CORRECT]
- **C.** Lower effort to low so Claude requests fewer calls per turn.
- **D.** Increase max_turns so Claude can spread the calls across several turns.

Correct answer: B. Correct (B): read-only tools can run concurrently while state-modifying ones run sequentially; custom tools default to sequential, and readOnlyHint in the annotations enables parallelism. A only auto-approves the calls without a permission prompt; it does not change concurrency. D does not reduce latency, it merely allows more turns. C changes reasoning depth, not how tools are executed.
Source: https://code.claude.com/docs/en/agent-sdk/agent-loop

Question 5 · D3 Claude Code · Claude Code · medium

In the middle of a long Claude Code conversation, the developer wants to try an alternative refactoring approach starting from the current history, without changing the original conversation, which must remain available. Select 2 suitable ways.

- **A.** **/branch, with an optional name for the new conversation.** [CORRECT]
- **B.** **claude --continue --fork-session, which copies the history into a new session ID.** [CORRECT]
- **C.** /compact, which summarizes the history to free up space.
- **D.** /clear, which starts with an empty context and keeps the previous history.
- **E.** claude --resume <name>, which reopens the session and continues from it.

Correct answer: A, B. Correct (A and B): /branch and --fork-session copy the history into a new ID and leave the original intact. C rewrites the same session's history as a summary, with no branching. D saves the previous session, but the new conversation does not inherit the history, so it does not start from the current point. E reopens the session under the same ID and appends messages to the existing conversation, changing the original.
Source: https://code.claude.com/docs/en/sessions

Question 6 · D5 Model Selection & Optimization · Technical Fundamentals · easy

A service catches a generic Exception when calling the API and decides what to do with checks like `"overloaded" in str(e)`. After a wording change in error messages, the handling stopped working. What is the best approach?

- **A.** **Catch the SDK's typed exception classes, most specific first, and decide by error type/status.** [CORRECT]
- **B.** Set max_retries=0 on the client so the SDK returns the error as raw JSON instead of raising an exception.
- **C.** Extract the HTTP code with a regex from the error message text.
- **D.** Add more substrings to the check to cover the new wordings.

Correct answer: A. Correct (A): the SDK raises typed exceptions per status/error type, and the docs say to catch those classes (most specific first) rather than string-match messages. D and C still depend on text, which can change. B is wrong: max_retries only configures automatic retries; the SDK still raises typed exceptions.
Source: https://platform.claude.com/docs/en/api/errors

Question 7 · D2 Applications & Integration · Claude Application Design · medium

You extract data from contracts with the API using structured outputs. Many contracts omit the end date and the jurisdiction, and the model sometimes invents plausible values for those fields. Which schema changes reduce the problem? Select 2.

A. Put every field in required so the model always returns a value for each one.

B. Add minLength: 1 to all strings to prevent empty values.

C. **Model the tipo_rescisao field as an enum that includes an explicit value such as "nao_informado", instead of a free string.** [CORRECT]

D. **Leave data_termino and foro out of required, making them optional, so that missing information can be represented.** [CORRECT]

E. Rely on structured outputs to guarantee that the extracted values are true to the document.

Correct answer: C, D. Correct (D and C): optional fields and an enum with an explicit "not informed" value give the model a legitimate output when the contract is silent, instead of forcing it to fill something in. A is wrong: requiring everything pressures the model to invent. B is wrong: minLength is listed among the unsupported string constraints in structured outputs and, even if it were, would push the model to fill something in. E is wrong: structured outputs enforce schema conformance (format); they do not make the content true to the document. Even hallucination-reduction techniques are described as reducing the problem, not eliminating it.
Source: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

Question 8 · D8 Tools & MCPs · Agentic Customization · easy

You created the /deploy skill, which runs the production deploy procedure. You want it to run only when you type /deploy, never on Claude's own initiative because the conversation seems to call for a deploy. What should you do?

A. **Set disable-model-invocation: true in the skill's frontmatter.** [CORRECT]

B. List only safe tools in the skill's allowed-tools.

C. Set context: fork in the frontmatter, to run in an isolated subagent.

D. Move the procedure to CLAUDE.md, which is only read when you ask.

Correct answer: A. Correct (A): with this option Claude does not load the skill automatically, and it only runs when invoked with /deploy (recommended for skills with side effects). D is wrong: CLAUDE.md is loaded every session. B is wrong: allowed-tools pre-approves tools, it does not prevent automatic invocation. C only changes where the skill executes.
Source: https://code.claude.com/docs/en/skills

Question 9 · D1 Agents & Workflows · Agent Construction with Claude · medium

An Agent SDK agent was configured with max_turns=5 to fix tests. In production, some runs end with no final text at all. Select 2 correct statements about this behavior and how to handle it.

A. The SDK automatically doubles max_turns and continues the loop when the limit is reached.

B. **The ResultMessage still carries session_id, so the session can be resumed with a higher limit to continue the work.** [CORRECT]

C. **The run ends with a ResultMessage of subtype error_max_turns and no result field; code must check the subtype before reading result.** [CORRECT]

D. Raising max_budget_usd also raises the turn limit, so a larger budget is enough to let the loop run longer.

E. max_turns counts only turns that call tools; the final text-only response never consumes a turn.

Correct answer: B, C. Correct (C and B): hitting the limit yields error_max_turns with no result, and every ResultMessage carries session_id for resuming. D is wrong: turns and budget are independent limits, each with its own error subtype. A is wrong: the SDK does not change the limit by itself. E is wrong: the count includes one extra turn for the final text-only response, and without it the run ends with no final response.
Source: https://code.claude.com/docs/en/agent-sdk/agent-loop

Question 10 · D2 Applications & Integration · Understanding Requirements · easy

A bank will launch a Claude-based support chatbot. Compliance requires that "every answer given to a customer be auditable". Which design decision best meets this requirement?

- **A.** **Persist, per response, the conversation id, prompt version, model used, the request-id returned by the API and the messages exchanged, with a defined retention policy** [CORRECT]
- **B.** Log only the final text shown to the customer, to reduce stored volume and the risk of retaining sensitive data
- **C.** Enable streaming, since the SSE events received by the customer's browser already form an auditable record of the interaction
- **D.** Rely on logs kept by Anthropic and request them from the provider whenever an internal or external audit takes place

Correct answer: A. Correct (A): auditability is a non-functional requirement that becomes traceable data under your control: which prompt/model produced the answer, the request-id (returned on every response) and the content, with a retention policy. D is wrong because audits depend on records you can produce yourself; the request-id is for support, not a replacement for your log. B is wrong because without prompt, model and context you cannot explain or reproduce why that answer was given. C is wrong because streaming is a delivery technique and creates no durable or trustworthy client-side record.
Source: https://platform.claude.com/docs/en/api/errors

Question 11 · D3 Claude Code · Claude Code · medium

The team wants a /release procedure that runs only when someone types the command (never by Claude's own decision) and whose long reference material (checklist and templates) does not take up context in every session. Select 2 suitable practices.

- **A.** Paste the full procedure and templates into the project's CLAUDE.md.
- **B.** Set user-invocable: false so the skill is only used deliberately.
- **C.** Use allowed-tools to control when Claude can trigger the skill.
- **D.** **Create .claude/skills/release/SKILL.md with disable-model-invocation: true.** [CORRECT]
- **E.** **Keep the checklist and templates in separate files inside the skill folder, referenced from SKILL.md.** [CORRECT]

Correct answer: D, E. Correct (D and E): disable-model-invocation: true makes the skill manual-only, and supporting files in the skill folder are read only when needed, keeping SKILL.md lean. B does the opposite: it hides the skill from the / menu and leaves it to Claude only. A loads the content in every session. C only pre-approves tools for the turn; it does not control who triggers the skill.
Source: https://code.claude.com/docs/en/skills

Question 12 · D5 Model Selection & Optimization · Cost and Token Management · hard

A nightly job sends 50k requests to the Message Batches API, all with the same 30k-token document as prefix and different questions. There is no urgency for responses. Select 3 correct practices to minimize cost.

- **A.** Inside the batch, send a request with max_tokens 0 to pre-warm the cache before the others.
- **B.** **Include identical cache_control blocks in every request of the batch to raise the chance of cache hits.** [CORRECT]
- **C.** Count on guaranteed hits, since the batch processes requests sequentially in order.
- **D.** **Consider the 1-hour TTL, since batches can take longer than 5 minutes to process.** [CORRECT]
- **E.** **Use the Batches API, which gives a 50% discount for non-urgent asynchronous work.** [CORRECT]

Correct answer: B, D, E. Correct (E, B and D): batch gives a 50% discount; identical cache_control and the 1-hour TTL raise the chance of hits in batches. C is wrong because batch requests can run concurrently and in any order, so hits are best-effort. A is wrong because max_tokens 0 is not supported inside a batch (the entry would likely expire before use).
Source: https://platform.claude.com/docs/en/build-with-claude/batch-processing

Question 13 · D2 Applications & Integration · Claude Application Design · medium

Your team enabled 12 plugins in Claude Code and notices sessions start with less context space, even on tasks that use few of them. Which statements about plugin cost are true? Select 2.

- **A.** **You can stop the cost without uninstalling by disabling the plugin with /plugin or with claude plugin disable.** [CORRECT]
- **B.** **The name and description of each skill Claude can invoke on its own sit in context every turn, even in sessions where nothing from the plugin is used.** [CORRECT]
- **C.** Uninstalling is the only way to remove a plugin's context cost.
- **D.** A plugin only affects the session when one of its skills is invoked; hooks and MCP servers stay inactive until then.
- **E.** The full text of all skills from enabled plugins is loaded at the start of every session.

Correct answer: A, B. Correct (B and A): descriptions of model-invocable skills occupy context all the time, and disabling the plugin stops the cost without uninstalling. E is wrong: full text loads only when the skill is used. D is wrong: an enabled plugin is part of every session, with hooks firing and MCP servers running. C contradicts A: disabling is enough.
Source: https://code.claude.com/docs/en/plugins/overview

Question 14 · D6 Prompt & Context Engineering · Context Engineering · hard

A dev is estimating how much of the context window a request for an agent consumes. The request has a system prompt, 12 tool definitions, a history with tool_result blocks, and the text Claude will generate in the turn. Which statements about what counts toward the context window are correct? Select 3.

- **A.** **The content of tool_result blocks in the history counts like any other message.** [CORRECT]
- **B.** **The text Claude generates in the turn, including thinking, counts toward the window.** [CORRECT]
- **C.** **Tool definitions count toward the window, just like the system prompt.** [CORRECT]
- **D.** Tokens read from a cached prefix (prompt caching) no longer occupy the window; caching only changes the price.
- **E.** Only user/assistant messages count; the system prompt and tool definitions are left out of the total.

Correct answer: A, B, C. Correct (B, C, A): everything in the request counts toward the window: the system prompt, every message in messages (including tool results), the tool definitions, and also the output generated in the turn, thinking included. D is wrong: cached prefixes still occupy the window; caching changes what you pay for those tokens, not whether they count. E is wrong: the system prompt and tool definitions do count, which makes tool definitions a real context cost in agents with many tools.
Source: https://platform.claude.com/docs/en/build-with-claude/context-windows

Question 15 · D7 Security & Safety · Identity, Secrets and Key Management · easy

A team wants to reduce the risk of leaking Claude API keys before any incident happens. Which two prevention practices are recommended? Select 2.

- **A.** Keep the same key without rotation until a leak is suspected.
- **B.** **Regularly scan repositories with a tool such as Gitleaks.** [CORRECT]
- **C.** Encode the key in base64 in the front-end bundle so it is not readable.
- **D.** Share a single team key over chat to speed up onboarding.
- **E.** **Load the key through an environment variable and list .env in .gitignore.** [CORRECT]

Correct answer: B, E. Correct (E, B): environment variables with .env in .gitignore keep the key out of the repository, and periodic repository checks catch accidentally committed secrets. C is wrong: base64 is encoding, not protection, and a key in client code is exposed. D is wrong: keys should not be shared, just like personal passwords. A is wrong: the recommendation is to rotate on a regular schedule, not only react to suspicion.
Source: https://support.claude.com/en/articles/9767949-api-key-best-practices-keeping-your-keys-safe-and-secure

Question 16 · D6 Prompt & Context Engineering · Prompt Engineering · medium

A prompt pastes three contracts as plain text separated by '---' lines, followed by the instructions. Claude sometimes attributes a clause to the wrong contract and mixes contract passages with the instructions. What is the best fix?

- **A.** Move the three contracts into the system prompt so they carry more authority than the instructions.
- **B.** **Wrap each contract in <document index="n"> with <source> and <document_content>, inside <documents>.** [CORRECT]
- **C.** Repeat the instructions after each contract so Claude does not confuse them with the text.
- **D.** Replace the '---' separators with uppercase banners like '=== CONTRACT 2 ==='.

Correct answer: B. Correct (B): consistent, descriptive XML tags separate instructions, context and data unambiguously; for multiple documents the recommended structure is <document> with <source> and <document_content> inside <documents>. D is still free text with no unambiguous delimiting and does not identify the source in a structured way. A does not fix the document mix-up and also blends data with instructions. C bloats the prompt and gives no clear boundaries between documents.
Source: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Question 17 · D7 Security & Safety · AI Application Security · easy

A support agent reads customer tickets and has the query_db tool with a database administrator credential and the send_email tool with no recipient restriction. The team fears a malicious ticket could make the agent exfiltrate data. What is the best approach?

- **A.** Politely ask, in the test tickets, that the agent ignore instructions contained in the customer text.
- **B.** Add 'never send data to third parties' to the system prompt and keep the current permissions.
- **C.** **A read-only credential on the needed tables, narrow tools, and no unnecessary secrets.** [CORRECT]
- **D.** Log all tool calls for auditing and review them weekly, keeping the permissions.

Correct answer: C. Correct (C): least privilege means a successful injection can do minimal damage: do not give access to unnecessary secrets and scope permissions as narrowly as possible. B relies only on model obedience, with no technical control. D is monitoring: a useful complementary layer, but it finds the leak after it happened and keeps the excessive permissions. A is not a security control; natural-language instructions do not limit what the credentials allow.
Source: https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks

Question 18 · D8 Tools & MCPs · Agentic Customization · medium

Your team has a lookup_customer function that queries the CRM. Today only one internal service, which calls the Messages API, uses it. Soon it will also be used in Claude Code, Claude Desktop and another app. Which reasons justify packaging it as an MCP server? Select 2.

- **A.** **Several hosts (Claude Code, Claude Desktop, other apps) can connect to the same server instead of each reimplementing the tool.** [CORRECT]
- **B.** Tools defined in application code cannot reach internal APIs; only MCP servers can.
- **C.** The Messages API only accepts tools that come from MCP servers.
- **D.** **The connection and authentication to the CRM live in one place, handled by the server, and are not duplicated in each client.** [CORRECT]
- **E.** A tool exposed via MCP is called by the model more accurately than a tool defined in the application code.

Correct answer: A, D. Correct (A and D): in the MCP architecture a host (Claude Code, Claude Desktop) creates one client per server, and a remote server serves many clients; authentication is handled at the server's transport layer instead of duplicated in each client (Claude Code's extensions overview likewise describes MCP as connection and authentication handled by the server). E is wrong: the protocol does not make calls more accurate; that depends on the tool's name, description and schema. B is wrong: a tool in your code can call any API your code can reach. C is wrong: tools declared in tools on the Messages API, executed in your code, work without MCP; for a single app, MCP would be overkill.
Source: https://modelcontextprotocol.io/docs/learn/architecture

Question 19 · D6 Prompt & Context Engineering · Output Handling · medium

An assistant answers questions about an annual report and sometimes confidently states figures that are not in the report. Which two measures help reduce and detect this? Select 2.

- **A.** **Run the same prompt several times and review the answers that disagree, as they may indicate hallucination.** [CORRECT]
- **B.** Instruct Claude to fill in gaps with general knowledge when the report lacks the data.
- **C.** Drop human validation once answers are restricted to the report.
- **D.** Ask Claude to report a confidence from 0 to 100 and automatically accept answers above 90.
- **E.** **Give explicit permission to answer that there is not enough information when the report lacks the data.** [CORRECT]

Correct answer: A, E. Correct (E, A): allowing 'I don't know' drastically reduces invented information, and comparing several runs (best-of-N) exposes inconsistencies that point to hallucination. D is wrong: the page does not propose self-reported confidence as verification; the listed techniques ground the answer in the source text (quotes, citations) or compare runs, and none of that is replaced by a number the model reports about itself. B goes the opposite way: the recommendation is to restrict to the provided content. C is wrong: the techniques reduce but do not eliminate hallucinations; critical information still needs validation.
Source: https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations

Question 20 · D2 Applications & Integration · Understanding Requirements · medium

A support chatbot requirement says only: "must be fast and always available". Before designing, the team must turn it into verifiable requirements. Select 2 suitable approaches.

- **A.** Set 100% availability as the goal, with no error response tolerated in any period
- **B.** **Define p95 time-to-first-token below an agreed value, under a specified number of concurrent sessions, measured in production** [CORRECT]
- **C.** Require average latency below the target, since the mean summarizes every user's experience well
- **D.** **Define an availability SLO (for example, % of requests served successfully per month) and the expected degraded behavior when the API returns 429 or 529** [CORRECT]
- **E.** Always pick the largest available model, since that guarantees fast responses and availability

Correct answer: B, D. Correct (B and D): percentiles with a load condition and measurement point make latency testable, and an SLO with explicit degraded behavior (queue, message, human handoff) makes availability realistic. C is wrong because the mean hides the long tail of slow responses. A is wrong because 100% is unrealistic and makes the solution expensive; the recommended practice is an SLO with an error budget. E is wrong because picking the largest model guarantees neither low latency nor availability; verifiable targets are what make this testable.
Source: https://sre.google/sre-book/service-level-objectives/

Question 21 · D2 Applications & Integration · Configuration Management · hard

The repository has a committed .claude/settings.json with permissions.allow ["Bash(npm run lint)"] and permissions.deny ["Read(./.env)"]. A new person clones the repo and opens Claude Code, without having trusted the folder yet. What happens?

- **A.** Neither applies until trust is granted, so .env can be read in the meantime.
- **B.** The allow rule applies right away, while deny only applies after trusting the folder, since it requires confirmation.
- **C.** **The deny rule applies right away; the allow rule only takes effect after trusting the folder, so lint still prompts.** [CORRECT]
- **D.** Both rules apply immediately, since the file is committed to the repository.

Correct answer: C. Correct (C): allow rules from the shared file wait for folder trust, while deny and ask rules apply immediately, keeping restrictions safe by default. D skips the trust step for allow. A would leave a security gap the design avoids. B inverts the logic: the permissive rule waits for confirmation, not the restrictive one.
Source: https://code.claude.com/docs/en/settings

Question 22 · D8 Tools & MCPs · MCP Server Development · hard

Your company wants a read-only MCP server over the ERP, used by hundreds of devs in Claude Code (and by other apps), identifying the user on every access. Which design fits best?

A. A remote server using SSE transport, as it is the most recommended for remote services.

B. A single stdio process on a central host, to which all devs connect at the same time.

C. **A remote server using Streamable HTTP transport, standard HTTP authentication (OAuth recommended by MCP for obtaining tokens) and clients connecting by URL.** [CORRECT]

D. A stdio server that each dev runs locally, with the ERP credential in an environment variable.

Correct answer: C. Correct (C): the MCP architecture describes remote servers using Streamable HTTP transport, which typically serve many clients and support standard HTTP authentication (bearer tokens, API keys, custom headers), with OAuth recommended for obtaining tokens. D spreads the ERP credential across every machine and does not serve other apps by URL. A is wrong: in Claude Code's MCP docs SSE is deprecated and HTTP is recommended for remote servers. B is wrong: local stdio servers typically serve a single MCP client.
Source: https://modelcontextprotocol.io/docs/learn/architecture

Question 23 · D5 Model Selection & Optimization · Cost and Token Management · medium

An internal tool calls Claude once every ~30 minutes with the same large prefix and cache_control at the default TTL. The bill went up after enabling caching, and the response shows high cache_creation_input_tokens and cache_read_input_tokens always 0. What is the best interpretation and action?

A. The prefix is too small, so nothing is read; raising max_tokens fixes it.

B. **The default TTL (5 minutes) expires before the next call (~30 minutes): every request pays the cache write and never reads; the action is to remove caching or evaluate the 1-hour TTL, which covers that gap between calls.** [CORRECT]

C. The usage fields are wrong on recent models; caching is working and billing will stabilize.

D. You need to add more breakpoints (up to 4) so the cache starts being read.

Correct answer: B. Correct (B): the default TTL is 5 minutes and writes cost more than normal input; without reuse inside the TTL there are only writes and no cheap reads. The 1-hour TTL is the documented option for longer gaps. C is wrong because the fields reflect what happened. D is wrong because more breakpoints do not prevent expiry. A is wrong because a prefix that is too short would not be cached at all (cache_creation_input_tokens would be 0, unlike what is observed) and max_tokens does not affect caching.
Source: https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Question 24 · D2 Applications & Integration · Systems Life Cycle · hard

A team keeps an extractor's system prompt in a repository and edits it in place under the label "v2". A "small" rewording changed the output format, broke the downstream parser and went to 100% of traffic at once, even though the offline evals passed. The team decides to adopt semantic versioning (SemVer) for its prompts. Which versioning and rollout approach is best?

A. Pick the version number by diff size: few changed lines mean PATCH, even if the output format changes

B. Release straight to 100% once offline evals pass, since they replace observation in production

C. Keep the "v2" label and overwrite it on every tweak, since Git history already lets you go back to any previous version

D. **Declare the output format as the prompt's public API, treat published versions as immutable (any change becomes a new version; broken format = MAJOR), gate with evals, then release as a canary with metrics and rollback** [CORRECT]

Correct answer: D. Correct (D): SemVer requires declaring a public API, forbids modifying the contents of a released version and reserves MAJOR for incompatible changes; applied to prompts, the contract the parser consumes is the output format. Evals are the gate, and a canary with monitoring and rollback (a gradual-rollout practice, not part of SemVer) limits whatever damage slips through. C is wrong because overwriting the label breaks immutability and no longer tells which prompt produced each production output. A is wrong because SemVer classifies by API compatibility, not diff size: a single line can break the contract. B is wrong because offline evals do not cover the real input distribution; the canary catches what slipped through.
Source: https://semver.org/

Question 25 · D1 Agents & Workflows · Agent Patterns & Frameworks · medium

A review subagent was defined in .claude/agents/reviewer.md, with description "Read-only reviewer; never modifies files" and no tools field. It still used Write and Edit. The team requires those two tools to be unavailable to it. Select 2 effective changes.

- **A.** **Add disallowedTools: Write, Edit to the subagent's frontmatter.** [CORRECT]
- **B.** Set isolation: worktree so edits never reach the main checkout.
- **C.** Reinforce in the description field that the subagent must never use Write or Edit.
- **D.** **Add tools: Read, Grep, Glob to the subagent's frontmatter.** [CORRECT]
- **E.** Set model: haiku, since a smaller model tends to edit less.

Correct answer: A, D. Correct (D and A): with no tools field, the subagent inherits every available tool; an allowlist (tools) or a denylist (disallowedTools) actually removes Write and Edit. C is wrong: description guides when Claude delegates to the subagent, it does not restrict tools. B isolates the directory, but Write and Edit remain available. E changes the model, unrelated to tool permissions.
Source: https://code.claude.com/docs/en/sub-agents

Question 26 · D5 Model Selection & Optimization · LLM Fundamentals · hard

A FAQ app on a Sonnet-tier model uses adaptive thinking. Monitoring shows thinking blocks on many trivial questions, adding latency and output cost. Adding "don't think, answer directly" to the system prompt did not reduce it reliably. What is the best approach?

- **A.** Set a small budget_tokens in thinking to cap reasoning at a few tokens.
- **B.** **Lower the effort level (output_config.effort), measure the quality impact, and only then add prompt guidance if still needed.** [CORRECT]
- **C.** Strengthen the system prompt ("NEVER think") and assume the model will always comply.
- **D.** Raise max_tokens so thinking finishes faster and frees the answer.

Correct answer: B. Correct (B): effort is the calibrated lever for how often and how deeply Claude thinks; prompt guidance is secondary, wording-sensitive and must be measured before shipping. C is wrong: per the docs, on models such as current Sonnet ones, asking in the prompt to think less does not reliably reduce thinking. A is wrong: the docs say you do not set a thinking budget on these models (thinking enabled with budget_tokens is rejected with a 400); cost is bounded by max_tokens and effort. D is wrong: max_tokens is only an output cap (thinking included) and does not make the model think less.
Source: https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost

Question 27 · D7 Security & Safety · Claude Hooks · medium

A team creates a PreToolUse hook that detects when Claude Code tries to read a .env file, prints a message to stderr, and ends with exit 1. The file is still read anyway. What is the best fix?

- **A.** Keep exit 1, which, being the conventional Unix failure code, already blocks the action.
- **B.** Move the logic to a PostToolUse hook, which runs after the tool has executed.
- **C.** **End the script with exit 2: in PreToolUse this blocks the tool call and the stderr becomes the block message.** [CORRECT]
- **D.** Print the message to stdout and end with exit 0, since the message alone cancels the tool call.

Correct answer: C. Correct (C): exit 2 is the blocking mechanism and, in PreToolUse, it blocks the call and stderr becomes the reason delivered to Claude. A is wrong: any exit code other than 0 and 2 is a non-blocking error and the tool call proceeds, even though 1 is the conventional failure code. B is wrong: PostToolUse runs after execution and cannot block (the tool already ran). D is wrong: exit 0 means success and the action proceeds unless output JSON denies it; text on stdout cancels nothing.
Source: https://code.claude.com/docs/en/hooks

Question 28 · D8 Tools & MCPs · Tool Implementation · hard

The get_invoice(invoice_id) tool runs in your backend, with strict: true and invoice_id as a string in the schema. Another tool returns, in a tool_result, the text of a third-party email that tries to induce Claude to request ANOTHER customer's invoice. Which statements are true? Select 2.

A. strict: true guarantees the invoice_id sent belongs to the authenticated customer.

B. **strict: true guarantees the tool's inputs match the declared schema.** [CORRECT]

C. Untrusted content should be placed in the system prompt, which is the safest place for it.

D. **Content in a tool_result coming from outside sources, such as emails, must be treated as untrusted, since it can embed instructions that try to redirect Claude (indirect prompt injection).** [CORRECT]

E. The email is inside a tool_result, so its content is trustworthy because it comes from your own backend.

Correct answer: B, D. Correct (B and D): strict guarantees schema conformance, and external content in a tool_result must be treated as untrusted (indirect injection). A is wrong: the schema only describes shape (here, a string); strict does not decide whether the invoice belongs to the customer. E is wrong: being inside a tool_result does not make content trustworthy; its origin is external. C is wrong: the guidance is to keep untrusted content inside tool_result blocks, not in system prompts.
Source: https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

Question 29 · D4 Eval, Testing & Debugging · Debugging & Error Handling · hard

Agent trace: on turns 1, 2 and 3 Claude calls search_docs with {"query":"rate limits"} and gets is_error=true: "Missing required 'repo' parameter". On turn 4 it apologizes and gives up. The search_docs schema requires repo, but the description is "Search docs." and repo has no description. What is the best fix?

A. Increase max_turns so Claude eventually gets the parameter right in more attempts.

B. Increase retries and backoff in the tool executor, since the integration is failing.

C. Switch to forced tool_choice on search_docs, since Claude picked the wrong tool.

D. **Expand the description of the tool and of the repo parameter (what it is, valid values) and enable strict: true, treating it as an under-specified definition.** [CORRECT]

Correct answer: D. Correct (D): the right tool was chosen; what is missing is information to fill repo. The docs recommend more detailed descriptions, and strict: true guarantees inputs match the schema, preventing missing parameters. C is wrong: there was no wrong tool choice, and forcing may be rejected on newer models. B is wrong: the executor never ran; the error is in the input. A is wrong: Claude already repeated and gave up, and more turns repeat the same error.
Source: https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

Question 30 · D8 Tools & MCPs · Tool Implementation · medium

Your app uses the Messages API with a client tool issue_refund, which moves money. You want a human to approve each refund before it happens. Which approach?

A. Turn issue_refund into a server tool, so that execution and approval are handled by Anthropic.

B. **In your code that handles the tool_use block, pause before executing, ask for approval and, if denied, return a tool_result with is_error true explaining the denial.** [CORRECT]

C. Let the handler execute the refund and, if the human disagrees afterwards, ask Claude to reverse it.

D. Write "always ask the user for approval before calling" in the tool description and trust Claude to comply.

Correct answer: B. Correct (B): with client tools your own code runs the tool, so that is where the control point lives; the denial returns as a tool_result with is_error so Claude can adapt. D is just a request in the prompt, with no guarantee. C performs the irreversible action before the decision. A is wrong: server tools are run internally by Claude/the API without passing through your code, so there is nowhere to insert your approval flow.
Source: https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

Question 31 · D2 Applications & Integration · Claude API Mechanics · medium

An app streams a response in which Claude calls a tool. The code runs json.loads on each partial_json received in input_json_delta events and throws on almost every event. What is the correct approach?

A. Catch the exception and use the last fragment that parsed successfully as the tool arguments

B. **Accumulate fragments per block index, parse on receiving content_block_stop, and only then run the tool** [CORRECT]

C. Run the tool as soon as the first field shows up in the fragment, using a regex, to reduce latency

D. Turn streaming off only during tool use, since there is no way to get the arguments JSON from streamed requests

Correct answer: B. Correct (B): the deltas are string fragments that only form valid JSON at the end of the block; accumulate them per index and parse at content_block_stop (or use the SDK helpers). C is wrong because executing with partial arguments may run an incorrect or incomplete action. A is wrong because the last successful parse is a truncated object, not the real arguments. D is wrong because streaming tool use is supported and the full JSON is obtained by accumulating the fragments.
Source: https://platform.claude.com/docs/en/build-with-claude/streaming

Question 32 · D1 Agents & Workflows · Agent Architecture · easy

A team uses a single Claude call to review code changes for vulnerabilities. The problem is variance: across repeated runs, real findings sometimes show up and sometimes disappear. Which workflow pattern addresses this problem?

A. **Voting: run the same review several times independently and flag the change when enough runs report a problem.** [CORRECT]

B. Routing: classify each change by language and send it to a specialized review prompt.

C. Prompt chaining: pass the review through a second prompt that rewrites and refines the report text.

D. Sectioning: split the diff into independent parts and process each part in parallel, each with a different prompt.

Correct answer: A. Correct (A): voting is the parallelization variation that runs the same task several times to get diverse outputs and aggregate them, which reduces single-call variance. D (sectioning) parallelizes distinct subtasks for speed or focus, but each part is still judged only once. C improves how the report reads, not detection reliability. B specializes prompts by input type, but each input is still judged only once.
Source: https://www.anthropic.com/engineering/building-effective-agents

Question 33 · D5 Model Selection & Optimization · LLM Fundamentals · medium

An application uses a model with a 1M-token context window. Each request sends a 300k-token prefix marked for prompt caching, plus the user question, with adaptive thinking on and max_tokens set. A colleague makes claims about context window usage. Select 2 correct statements.

A. **Generated thinking tokens count toward the context window and are part of the max_tokens limit.** [CORRECT]

B. The window limits only input; output tokens are counted against a separate limit.

C. **The 300k cached prefix tokens still occupy the context window; caching changes their price, not the space they take.** [CORRECT]

D. Tokens read from cache do not count toward the window because they are not reprocessed as normal input.

E. If the input alone exceeds the window, the API silently truncates the oldest messages and responds normally.

Correct answer: A, C. Correct (C and A): the window covers system prompt, messages, tools and the generated output (including thinking); cached tokens still take space and only their price changes. D is wrong because caching does not reduce window usage. E is wrong because input larger than the window returns a 400 error (prompt is too long), with no truncation. B is wrong because the window covers input and output together.
Source: https://platform.claude.com/docs/en/build-with-claude/context-windows

Question 34 · D5 Model Selection & Optimization · Technical Fundamentals · medium

Before sending large prompts, your internal API calls the token counting endpoint to decide routing and check limits. Select 2 correct statements about this endpoint.

- **A.** **It returns an estimate; the actual input token count at message creation may differ by a small amount.** [CORRECT]
- **B.** It returns the exact number that will be billed, always identical to the real response usage.
- **C.** **It is free to use, subject to its own requests-per-minute limits, independent of message creation limits.** [CORRECT]
- **D.** It honors cache_control blocks, so the result already reflects the cache read discount.
- **E.** It accepts the same request as the Messages API, including server tools such as web search.

Correct answer: A, C. Correct (C and A): counting is free with a separate rate limit and is an estimate. B is wrong because there is no guarantee of equality with billed usage. D is wrong because caching only occurs on real message creation. E is wrong because server tools (web search, code execution, etc.) return invalid_request_error on this endpoint.
Source: https://platform.claude.com/docs/en/build-with-claude/token-counting

Question 35 · D1 Agents & Workflows · Agent Patterns & Frameworks · easy

An Agent SDK agent received "always write release notes in Portuguese" in its initial prompt. In long sessions, after compact_boundary events, it starts writing in English. What is the best fix?

- **A.** Set effort to max so the model remembers the initial instructions better.
- **B.** **Move the rule into a CLAUDE.md loaded through setting_sources, which is re-injected on every request.** [CORRECT]
- **C.** Raise max_budget_usd so the original context is not summarized.
- **D.** Resume the session with session_id on every turn to restore the original prompt.

Correct answer: B. Correct (B): compaction replaces older messages with a summary, and instructions from early in the conversation may not survive; persistent rules belong in CLAUDE.md, re-injected on every request. C does not control compaction: a budget caps spend. A changes reasoning depth, not what stays in context. D restores the already compacted context, with the summary in place of the original messages.
Source: https://code.claude.com/docs/en/agent-sdk/agent-loop

Question 36 · D5 Model Selection & Optimization · Technical Fundamentals · medium

In an agent loop, a response arrives with stop_reason "max_tokens" and the last content block is a tool_use whose JSON input looks cut off mid-way. What should you do?

- **A.** **Treat the response as incomplete and retry the request with a higher max_tokens instead of running the tool.** [CORRECT]
- **B.** Send the partial tool_use back as a tool_result and ask the model to complete the JSON.
- **C.** Run the tool with the partial input, assuming the fields that arrived are enough.
- **D.** Ignore the stop_reason, since the SDK automatically completes truncated tool calls.

Correct answer: A. Correct (A): max_tokens means the response hit the cap; with a truncated tool_use the recommendation is to retry with a higher limit. C is wrong because incomplete input may trigger wrong actions. B is wrong because tool_result is the output of an already-made call, not a channel for completing JSON. D is wrong because the docs say to retry with a higher max_tokens and do not describe the SDK completing a truncated tool_use.
Source: https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons

Question 37 · D2 Applications & Integration · Software Engineering Foundations · medium

A Node service processes 2,000 documents by firing all API calls with Promise.all and receives a flood of 429 errors. Select 3 suitable fixes.

- **A.** **Ramp load up gradually instead of jumping from zero to peak, which also avoids acceleration limits** [CORRECT]
- **B.** Retry the request immediately in a loop, until one of the attempts goes through
- **C.** Add independent retries at every layer (queue, service and client), so each failure gets more chances to pass
- **D.** **Limit concurrency with a worker pool or semaphore, sized below the organization's rate limits** [CORRECT]
- **E.** **Apply retry with exponential backoff and jitter, honoring the retry-after header when present and with a maximum number of attempts** [CORRECT]

Correct answer: A, D, E. Correct (D, E and A): bounded concurrency avoids exceeding the rate; exponential backoff honoring retry-after spaces new attempts (jitter is a general practice to avoid synchronizing them); and a gradual ramp avoids acceleration limits, as the rate limits page describes. B is wrong because retrying without waiting worsens overload and keeps getting 429. C is wrong because the official SDK already retries transient failures automatically (twice by default, honoring retry-after); stacking independent retries in queue, service and client multiplies attempts per failure and aggravates overload.
Source: https://platform.claude.com/docs/en/api/rate-limits

Question 38 · D1 Agents & Workflows · Agent Construction with Claude · easy

An Agent SDK agent will carry out a large refactor. The team wants it to read the code and propose a plan for human approval before modifying any file. Which permission_mode fits?

- **A.** **plan, where Claude explores and plans, and edits made with its file tools go to the canUseTool callback.** [CORRECT]
- **B.** dontAsk, which never prompts, so nothing unreviewed ever gets executed.
- **C.** bypassPermissions, which runs calls without asking for approval and is recommended for local testing.
- **D.** acceptEdits, which approves edits automatically to speed up iteration.

Correct answer: A. Correct (A): plan mode is designed to produce a plan first, and edits go through canUseTool even if an allow rule would cover them. D approves edits without review, the opposite of the request. B targets headless agents with nobody to approve: it denies what would prompt instead of presenting a plan. C skips the checks and is only suited to isolated environments such as CI or containers.
Source: https://code.claude.com/docs/en/agent-sdk/agent-loop

Question 39 · D2 Applications & Integration · Claude Application Design · easy

In your team's Claude Code there are two instructions: (1) "tests run with pnpm test and API handlers live in src/api/handlers/"; (2) "short answers, with little explanation", which some devs want to turn off while learning the code. Where should each go?

- **A.** (1) in a skill with disable-model-invocation; (2) in a SessionStart hook.
- **B.** **(1) in CLAUDE.md; (2) in an output style, which can be switched whenever wanted.** [CORRECT]
- **C.** (1) in an output style; (2) in CLAUDE.md.
- **D.** Both in CLAUDE.md, since it loads every session and covers any kind of instruction.

Correct answer: B. Correct (B): project facts and conventions (build commands, structure) are permanent CLAUDE.md context; the format and tone of responses are the job of an output style, which can be switched. C swaps the roles. D loses the ability to toggle the style, since CLAUDE.md is always loaded. A uses mechanisms for other purposes: an on-demand skill does not guarantee conventions are always in context, and hooks are for event automation, not tone.
Source: https://code.claude.com/docs/en/features-overview

Question 40 · D5 Model Selection & Optimization · Model Selection and Tradeoffs · easy

A team will migrate from an older-generation model to a newer-generation tier while keeping the same prompts. Their cost budget and "fits in the window" checks use token counts measured on the old model. What is the best approach?

A. Reuse the counts, since the number of tokens for a text is the same on every Claude model.

B. Re-evaluate only if the price per million tokens changes; otherwise the budget stays valid.

C. **Recount the prompts with the token counting endpoint using the target model and redo cost and window estimates, plus run evals.** [CORRECT]

D. Assume newer models always produce fewer tokens for the same text and lower the budget.

Correct answer: C. Correct (C): newer models may use a different tokenizer, producing more tokens for the same text; the docs say to recount with the target model because billing and window reflect that tokenizer. A is wrong because the count depends on the model. B is wrong because the same unit price still multiplies more tokens. D is wrong because the direction may be the opposite (more tokens).
Source: https://platform.claude.com/docs/en/build-with-claude/token-counting

Question 41 · D2 Applications & Integration · Software Engineering Foundations · easy

A service uses output_config.format with a JSON schema and writes Claude's output straight to the database. In production, parsing sometimes fails on responses cut off midway. What is the best approach?

A. Apply a function that "fixes" truncated JSON by closing braces and quotes, and write the repaired result

B. Treat the output as always valid, since structured outputs guarantee the schema on any response

C. Remove the schema and start the assistant turn with "{" to force JSON format on the newest models

D. **Check stop_reason before parsing, handle max_tokens (retry with a higher limit) and refusal as separate cases, and validate at the boundary before writing** [CORRECT]

Correct answer: D. Correct (D): structured outputs guarantee schema conformance, except when stop_reason is max_tokens (incomplete output) or refusal (the refusal takes precedence over the schema); so you check stop_reason and validate at the boundary before persisting. B is wrong because those two cases break the guarantee. C is wrong because assistant-message prefill is not supported on 4.6 and later models and returns 400. A is wrong because repairing truncated JSON writes possibly incomplete data as if it were valid.
Source: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

Question 42 · D2 Applications & Integration · Claude Application Design · easy

In a long Claude Code session you ask for a formatting tweak. Claude gets it wrong, you correct it, and it gets the same thing wrong again. What is the best next action?

A. **Run /clear and start with a single new prompt that already folds in what you learned from the attempts.** [CORRECT]

B. Make a third correction, writing the instruction in capital letters for emphasis.

C. Add the sentence to CLAUDE.md in ALL CAPS with IMPORTANT and keep going in the same session.

D. Keep correcting in the same conversation, since more history of attempts gives the model more context.

Correct answer: A. Correct (A): after two failed corrections the conversation already contains two failed attempts that Claude keeps re-reading; clearing and rewriting the prompt works better. B and D only grow that history. C is wrong: current models follow a plainly worded line, and excess emphasis tends to make Claude over-apply the rule.
Source: https://code.claude.com/docs/en/best-practices

Question 43 · D2 Applications & Integration · Claude API Mechanics · medium

A Message Batches API job with 5,000 requests finished with 4,700 succeeded, 200 errored and 100 expired results. Select 3 correct practices to consolidate and recover the work.

- **A.** **Resubmit expired items and server-error items in a new batch, and first fix items with invalid_request_error** [CORRECT]
- **B.** Match each result to its input by line position in the results file, since it follows submission order
- **C.** **Match each result to its source input by custom_id, since result order is not guaranteed** [CORRECT]
- **D.** **Count on errored and expired items not being billed, so reprocessing them does not pay twice for the same work** [CORRECT]
- **E.** Resubmit the entire 5,000-item batch to ensure all results come from the same processing run

Correct answer: A, C, D. Correct (C, A and D): custom_id is the correlation key; expired or transient-error items can go into a new batch, while invalid_request requires fixing the input; and errored/expired requests are not billed. B is wrong because results can come back in any order. E is wrong because reprocessing the 4,700 successful items wastes cost and time for no gain.
Source: https://platform.claude.com/docs/en/build-with-claude/batch-processing

Question 44 · D2 Applications & Integration · Claude API Mechanics · easy

In a chat with Claude, the dev calls the Messages API on each turn sending only the new user message. The assistant "forgets" everything said before. What is the most appropriate fix?

- **A.** Increase max_tokens, so the model can retain more context from one call to the next
- **B.** Send only the user's earlier turns on each call, omitting the assistant's replies to save tokens
- **C.** Reuse the same request-id header on every turn, so the API associates the calls with the same conversation
- **D.** **Store the conversation's message list and send it in full on every call, appending the assistant's reply and the new user turn** [CORRECT]

Correct answer: D. Correct (D): the Messages API is stateless, so the full history (user and assistant turns) must be sent on every request; your application holds the state. C is wrong because the request-id is a response identifier used for support and does not link calls. A is wrong because max_tokens limits output, it creates no memory across requests. B is wrong because without the assistant's replies the context is incomplete and the model loses what it said and concluded itself.
Source: https://platform.claude.com/docs/en/build-with-claude/working-with-messages

Question 45 · D6 Prompt & Context Engineering · Output Handling · easy

A triage service uses output_config.format with a JSON schema whose "category" field is an enum with the values "Billing", "Technical support" and "Other". The code compares the received category by exact equality and occasionally falls into the "unknown" branch because, for example, "Technical Support" arrived. What is the best fix?

- **A.** **Compare the value case-insensitively (normalizing first) and avoid enum values that differ only in capitalization.** [CORRECT]
- **B.** Abandon structured outputs and ask for free-text JSON, losing the structure conformance the schema guarantees.
- **C.** Assume the schema guarantees the exact enum spelling and treat the mismatch as an API bug to report.
- **D.** Treat it as a transient error and retry the request until the value comes back exactly as in the schema.

Correct answer: A. Correct (A): the structured outputs docs warn that capitalization of string enum and const values is not guaranteed, and recommend comparing case-insensitively and avoiding values that differ only in capitalization. D is wrong: the behavior is documented, not transient, and retrying only burns calls. B is wrong: it gives up the schema's structural guarantee without fixing the fragile comparison. C is wrong: it contradicts the docs, which explicitly caveat capitalization.
Source: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

Question 46 · D6 Prompt & Context Engineering · Context Engineering · medium

A pipeline generates contract summaries. The team wants to log the draft, evaluate the review of the draft against criteria and, between steps, decide whether to proceed to refinement or divert to human review. What is the best approach?

- **A.** **Split into sequential calls (draft, review, refine) and branch on the output of each step.** [CORRECT]
- **B.** A single prompt that asks for draft, review and final version, returning only the final version to save tokens.
- **C.** Raise the effort of the single call so the internal review is more rigorous, and skip logging the steps.
- **D.** Rely on adaptive thinking to do the three steps internally, since the model reasons through multiple steps on its own.

Correct answer: A. Correct (A): prompt chaining is useful when you need to inspect intermediate outputs or enforce a pipeline structure; in the self-correction pattern each step is a separate call, which allows logging, evaluating and branching. B hides draft and review, making the diversion impossible. D is wrong: the model can reason through several steps, but internal reasoning gives your code no point to choose the path. C creates no inspection point; higher effort does not replace logging the steps.
Source: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Question 47 · D2 Applications & Integration · Software Engineering Foundations · easy

A web chat shows Claude's reply token by token. The user sends each question through a normal POST request, and the server only needs to push text back during generation. Which server-to-browser transport is the best choice?

- **A.** Call Anthropic's API directly from the browser with the key embedded in JavaScript, removing the intermediate server
- **B.** Polling an endpoint every 100 ms, fetching the newly accumulated tokens
- **C.** WebSocket, because streaming responses requires a bidirectional channel between browser and server
- **D.** **Server-Sent Events over HTTP, since the flow is server-to-client only and the browser reconnects automatically** [CORRECT]

Correct answer: D. Correct (D): SSE is one-way, runs over HTTP and has automatic reconnection, which is enough when user input goes through POST. C is wrong because the bidirectional need does not exist here; WebSocket only pays off when the client also sends frequent messages on the same channel. B is wrong because polling adds needless traffic and latency for a continuous flow. A is wrong because exposing the API key in the browser is a serious security flaw.
Source: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

Question 48 · D2 Applications & Integration · Configuration Management · easy

Your team wants everyone in the repository to start Claude Code with the same default model, and for changes to that choice to go through code review and be revertible. Which approach?

- **A.** Each dev runs /model and saves the choice, which is written to each person's ~/.claude/settings.json.
- **B.** Write a line in the project's CLAUDE.md asking Claude to use the desired model.
- **C.** **Set the model key in .claude/settings.json and commit it to the repository.** [CORRECT]
- **D.** Set the model key in each dev's .claude/settings.local.json.

Correct answer: C. Correct (C): the project settings.json is shared through git, so the choice goes through a PR and has history for rollback. A stays in each person's own settings, with no standardization. B is wrong: CLAUDE.md is context, not configuration, and does not switch the session model. D is a personal file, outside version control.
Source: https://code.claude.com/docs/en/settings

Question 49 · D6 Prompt & Context Engineering · Prompt Engineering · easy

Claude's output goes to an SMS gateway that breaks on markdown. The system prompt says 'NEVER use markdown' and asterisks still show up now and then. What is the best improvement?

- **A.** Move the prohibition to the end of each user message and remove it from the system prompt.
- **B.** Add a rule saying formatting mistakes will be penalized and the response will be rejected.
- **C.** Write the same prohibition three times in uppercase to reinforce its priority.
- **D.** **Explain the reason: the reply goes out by SMS and markdown shows up literally to the user.** [CORRECT]

Correct answer: D. Correct (D): giving the context or motivation behind an instruction helps Claude understand your goal and generalize to cases you did not foresee, instead of following a bare prohibition. C only raises the emphasis, without saying why. A changes where the rule lives without adding information. B is a threat with no context; it does not tell Claude what the real problem with the SMS channel is.
Source: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Question 50 · D7 Security & Safety · AI Application Security · hard

An agent that summarizes web pages and emails from unknown senders is going to production. Which three practices are consistent with defending against indirect injection? Select 3.

- **A.** **Test before deployment with documents, emails and tool outputs that contain deliberate injection attempts.** [CORRECT]
- **B.** Insert the page text into the system prompt so Claude gives the retrieved content more weight.
- **C.** **Give the agent only the tools and data it needs, running them in an isolated environment with restricted permissions.** [CORRECT]
- **D.** Skip adversarial testing, since Claude is inherently resilient to this kind of attack.
- **E.** **Continuously analyze outputs for signs of successful injection and use that to refine prompts and filters.** [CORRECT]

Correct answer: A, C, E. Correct (C, A, E): red-teaming with hostile content before deployment, continuous monitoring and least privilege (sandbox, no unnecessary secrets) form layers that limit damage even if an injection gets through. B is wrong: third-party content does not belong in the system prompt; it goes in tool_result, treated as untrusted data. D is wrong: the model's resilience does not replace the extra layers or the testing.
Source: https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks

Question 51 · D2 Applications & Integration · Claude Application Design · easy

You want to try a plugin (with skills and hooks) only in this repository, without affecting your other projects or the teammates who use the same repo. Which install scope should you pick?

- **A.** User scope, which enables it for you in every project on this machine.
- **B.** Managed scope, set by the organization for all developers.
- **C.** Project scope, recording it in the committed .claude/settings.json and asking teammates to ignore it.
- **D.** **Local scope, which enables it for you only in this repository.** [CORRECT]

Correct answer: D. Correct (D): local scope applies only to you in this repository. A would reach all your projects. C enables the plugin for everyone working in the repo through committed settings, and asking them to ignore it is not a control. B is organization policy, not a personal test choice.
Source: https://code.claude.com/docs/en/plugins/overview

Question 52 · D1 Agents & Workflows · Agent Architecture · easy

A three-call pipeline extracts invoice fields, reconciles them with the ERP and drafts a collection email. Sometimes extraction returns empty required fields and the final email comes out confident but wrong. What is the best approach?

- **A.** Add "be careful if the data is incomplete" to the email prompt and keep the rest of the flow unchanged.
- **B.** Merge the three steps into one larger prompt so the model sees the whole task at once.
- **C.** **Add a programmatic check (gate) after extraction that validates the required fields and stops or retries the step before moving on.** [CORRECT]
- **D.** Replace the pipeline with an autonomous agent that decides on its own which steps to run and in what order.

Correct answer: C. Correct (C): in prompt chaining, programmatic checks (gates) between steps keep the process on track; the failure is stopped where it originates, deterministically. B removes the control points and makes debugging harder. A relies on the final model noticing missing data that it does not receive as an error. D swaps a known sequence of steps for autonomy, adding cost and unpredictability without addressing the cause.
Source: https://www.anthropic.com/engineering/building-effective-agents

Question 53 · D1 Agents & Workflows · Agent Construction with Claude · hard

An agent built with the Claude Agent SDK exposes a custom get_order tool through an in-process MCP server registered in mcpServers under the key "shop". Claude sees and uses the tool, but in unattended runs every call stalls waiting for a permission approval. Which change lets the calls run without a prompt?

- **A.** Mark the tool with readOnlyHint: true in its annotations, since read-only tools skip approval.
- **B.** Rewrite the tool description to say it is safe so Claude does not ask for approval.
- **C.** Pass tools: ["get_order"] in the options so the tool is both available and approved.
- **D.** **Add the fully qualified name mcp__shop__get_order (or the wildcard mcp__shop__*) to allowedTools.** [CORRECT]

Correct answer: D. Correct (D): tools from an in-process MCP server appear as mcp__{server}__{tool}, and listing that name (or the server wildcard) in allowedTools approves them without a prompt. C is wrong: the tools option filters only built-in tools (MCP tools are unaffected) and grants no permission. A is wrong: readOnlyHint is metadata that only controls whether the call can run in parallel with other read-only calls; annotations are not enforcement. B is wrong: the description lets Claude decide when to call the tool and does not change the permission flow.
Source: https://code.claude.com/docs/en/agent-sdk/custom-tools

Score by domain

Domain	Questions	Correct	%
D1 Agents & Workflows	8		
D2 Applications & Integration	17		
D3 Claude Code	2		
D4 Eval, Testing & Debugging	1		
D5 Model Selection & Optimization	9		
D6 Prompt & Context Engineering	6		
D7 Security & Safety	4		
D8 Tools & MCPs	6		
Total	**53**		

Reference: the real exam requires a scaled score of 720 (scale 100–1,000). As a rough estimate, ~72% correct (≥ 39 of 53) indicates passing level. This is not the official conversion, since scores are scaled.

Topics to review

For each wrong answer, note the domain/skill (the 'Skill' column in the summary) and re-read the matching official documentation. Prioritize the heaviest domains: Applications & Integration (33.1%) and Model Selection & Optimization (16.8%).

_____

_____

_____